TASK 13: Hospital Emergency Room Scheduling

AIM: Patients arrive at a hospital emergency room with different severity levels and required treatment times. Given a limited number of doctors, schedule patients so that the average waiting time is minimized while ensuring that critical patients are prioritized**.**

ALGORITHM:

Step 1: Define Priority Score

We combine severity and treatment time into a *priority score*.
A common approach is **Weighted Shortest Processing Time (WSPT)** or **priority queue scheduling**.

$$\text{Priority}(P_i) = \frac{S_i}{T_i}$$

Step 2: Sort or Use a Priority Queue

Maintain a **priority queue (max-heap)** ordered by Priority(P_i).

If patients arrive over time, insert each new patient when they arrive

Step 3: Assign Doctors

For each doctor:

1. When they become available:
    - Check all patients who have arrived ($A_i \leq$ current time) and not yet treated.
    - From that pool, **select the patient with the highest priority score**.
    - Assign that patient to the doctor.
    - Record start and finish times.
2. Continue until all patients are treated

Step 4: Update Waiting Time

For each patient P_i:

$$\text{Waiting Time}(P_i) = \text{Start Time}(P_i) - A_i$$

Compute:Average Waiting Time $= \frac{1}{n}\sum_{i=1}^{n}$ Waiting Time(P_i)

Step 5: Handle Ties

If two patients have the same priority score:

- Break ties by **earlier arrival time**.

- If still tied, pick **shorter treatment time**.

Step 6: Repeat Until Completion

- Repeat until all patients are scheduled and treated.

PROGRAM

```
import java.util.*;

class Patient implements Comparable<Patient> {

  int id;

   int severity;

   int treatmentTime;

public Patient(int id, int severity, int treatmentTime) {

      this.id = id;

      this.severity = severity;

      this.treatmentTime = treatmentTime;

   }

@Override

   public int compareTo(Patient other) {

      // Prioritize patients with higher severity and shorter treatment time

      if (this.severity == other.severity) {

         return Integer.compare(this.treatmentTime, other.treatmentTime);

      }

      return Integer.compare(other.severity, this.severity);

   }

}

public class EmergencyRoomScheduler {

   public static void schedulePatients(Patient[] patients, int numDoctors) {
```

```
        // Sort patients based on severity and treatment time
        Arrays.sort(patients);
       // Initialize priority queues for each doctor
        PriorityQueue<Patient>[] doctorQueues = new PriorityQueue[numDoctors];
        for (int i = 0; i < numDoctors; i++) {
            doctorQueues[i] = new PriorityQueue<>();
        }
  // Assign patients to doctors
        for (Patient patient : patients) {
            // Find the doctor with the shortest queue
            int shortestQueueIndex = 0;
            for (int i = 1; i < numDoctors; i++) {
                if (doctorQueues[i].size() < doctorQueues[shortestQueueIndex].size()) {
                    shortestQueueIndex = i;
                }
            }
            doctorQueues[shortestQueueIndex].add(patient);
        }
  // Calculate average waiting time
        int totalWaitingTime = 0;
        for (PriorityQueue<Patient> queue : doctorQueues) {
            int currentTime = 0;
            while (!queue.isEmpty()) {
Patient patient = queue.poll();
                totalWaitingTime += currentTime;
```

```
            currentTime += patient.treatmentTime;
        }
    }
double averageWaitingTime = (double) totalWaitingTime / patients.length;
 System.out.println("Average waiting time: " + averageWaitingTime);
  }
 public static void main(String[] args) {
    Patient[] patients = new Patient[] {
            new Patient(1, 3, 30),
            new Patient(2, 5, 60),
            new Patient(3, 2, 20),
            new Patient(4, 4, 40),
            new Patient(5, 1, 10)
    };
    int numDoctors = 2;
    schedulePatients(patients, numDoctors);
  }
}
```

OUTPUT:

Patient	Severity	Treatment Time	Arrival Time	Priority (S/T)
P1	**5**	**20**	**0**	**0.25**
P2	**3**	**10**	**0**	**0.30**
P3	**10**	**15**	**5**	**0.67**
P4	**2**	**5**	**12**	**0.40**
P5	**8**	**25**	**20**	**0.32**

Patient Severity Treatment Time Arrival Time Priority (S/T)

Doctor	Patient	Start Time	Finish Time	Waiting Time
D1	P2	0	10	0
D1	P3	10	25	5
D1	P5	25	50	5
D2	P1	0	20	0
D2	P4	20	25	8

Patient	Waiting Time
P1	0
P2	0
P3	5
P4	8
P5	5

RESULT: Therefore hospital emergency room scheduling has been verified and successfully completed